



IE2092: Introduction to Cyber Security
2nd Year, 2nd Semester

Kernel-Level Access Control: Advanced Security in Linux Architectures

Sri Lanka Institute of Information Technology

IT24102268

Aluthge M.N

Bachelor of Science Special Honors Degree in Information Technology

April 10, 2026

Table of Contents

1. Introduction to the Topic

- 1.1 The Shift to Cloud-Native and Containerized Environments
- 1.2 The Critical Role of Kernel-Level Security
- 1.3 Report Objectives

2. Evolution of the Topic

- 2.1 Foundational Access Control Models in Operating Systems (*<- Your new addition*)
- 2.2 Discretionary Access Control (DAC): Mechanisms and Fatal Flaws
- 2.3 The Shift to Mandatory Access Control (MAC) and Zero-Trust
- 2.4 The Linux Security Module (LSM) Architecture (Fixing TOCTOU Vulnerabilities)
- 2.5 SELinux: Complex Label-Based Type Enforcement
- 2.6 AppArmor: Usability and Path-Based Confinement
- 2.7 The eBPF Paradigm: Programmable, Real-Time Kernel Security

3. Future Developments in the Area

- 3.1 Defeating Code Reuse: Hardware-Assisted Execution Protections (PAC, BTI)
- 3.2 Hardware-Assisted eBPF Isolation (The HIVE Framework)
- 3.3 Confidential Computing in Cloud Architectures (Intel TDX / AMD SEV-SNP)
- 3.4 In-Kernel Machine Learning and Automated Anomaly Detection

4. Conclusion

- 4.1 Summary of the Evolution
- 4.2 Recommendations for Future Research

Abstract

The rapid transition from legacy monolithic architectures to cloud-native, containerized microservices has fundamentally redefined the enterprise security perimeter. Because containers share the underlying host operating system, traditional network edge firewalls are no longer sufficient, shifting the frontline of defense directly to the highly privileged kernel level. This report critically analyzes the architectural evolution of operating system access controls and kernel protection mechanisms. It traces the historical progression from the flawed, identity-centric Discretionary Access Control (DAC) models of early Unix systems to the rigorous, zero-trust enforcement of Mandatory Access Control (MAC) frameworks such as SELinux and AppArmor, which successfully mitigated race conditions through the Linux Security Module (LSM) architecture. Furthermore, it examines the paradigm-shifting introduction of the Extended Berkeley Packet Filter (eBPF), which enables highly granular, real-time programmable security within the kernel space. Finally, the report investigates emerging future developments necessary to secure modern cloud infrastructure against sophisticated advanced persistent threats (APTs) and zero-day vulnerabilities. Major outcomes highlight the critical necessity of transitioning from software-only defenses to hardware-assisted architectures, specifically emphasizing Pointer Authentication Codes (PAC), Branch Target Identification (BTI), hardware-assisted eBPF isolation (HIVE), Confidential Computing enclaves, and the integration of autonomous Machine Learning for in-kernel anomaly detection. Ultimately, this report concludes that the future of operating system security relies on a synergistic "Zero-Trust-on-Silicon" model, combining programmable kernel monitors with immutable hardware enforcement.

1. Introduction to the Topic

1.1 The Shift to Cloud-Native and Containerized Environments

Monolithic applications on dedicated physical servers or big, static virtual machines had retained their footing within enterprise IT infrastructure for decades. The security perimeter in these legacy environments was rather easy to establish. Organizations essentially built their own digital moat when they relied on a heavy perimeter firewall around the boundary of the corporate network and trusted everything running inside that line. There has been an entire paradigm shift in the last decade towards a cloud-native architecture. In the span of a few years, modern organizations have forsaken monolithic architectures in favor of microservices, intentionally splitting huge applications into hundreds and even thousands of lightweight containers that can be ephemeral.

Distributed microservices can spin up, perform a task, communicate across the cluster, and terminate all in seconds with technologies like Docker and orchestration platforms like Kubernetes. Although this architectural transition empowers immense scalability and deployment agility, it annihilates the traditional security perimeter. When an enterprise app runs on constantly changing nodes that are networked via complex, software-defined virtual networks, the legacy model of using edge firewalls devolves into complete irrelevance.

Since containers fundamentally share the exact same underlying operating system kernel with their host, the attack surface has increased exponentially. If a single, low-privileged web container is breached, the attacker is already "inside the moat." This has forced the frontline of modern cybersecurity to shift rapidly inward. This means that the industry no longer just needs to defend the edge of a network, but must now instead protect the core operating system, making kernel-level security the foundation that all modern cloud infrastructure must be built around.

1.2 The Critical Role of Kernel-Level Security

Over every Linux machine, the magic number is the kernel, reigning as the absolute winner of resources. Hundreds of microservices where each believes it is running on its own server, but everyone actually communicating with the same host kernel within a containerized cloud environment. The kernel runs in the very privileged Ring 0 mode, granting it unrestricted, superuser-level access to the CPU, physical memory and hardware peripherals.

If an attacker does get execution on a specific space because they somehow leverage a vulnerability to gain root level execution, it's completely game over. Software firewalls, namespace isolation, container boundaries, and default user permissions go immediately out of the window because the attacker controls the engine that enforces those rules. Thus, protecting this layer is not just an operational best practice; it is the one and only requirement of a modern age enterprise cybersecurity plan. If the kernel falls down, then the whole cloud infrastructure drops along with it.

1.3 Report Objectives

This report seeks to analyze the architecture of operating system access controls and kernel protection mechanisms over time. The main aim is to follow the evolution of this technology, from imperfect, monolithic discretionary models of early Unix systems through strict Mandatory Access Control (MAC) enforcement up to highly dynamic, programmable enforcement mechanisms such as eBPF deployed in modern cloud architectures. In addition, it describes the leading edge of future security advancements by assessing how hardware-backed confidential computing and autonomous machine learning algorithms are already transforming infrastructure defense within the next decade.

2. Evolution of the Topic

2.1 Foundational Access Control Models in Operating Systems

The advances kernel-level security frameworks are eBPF and SELinux, it is essential to first understand the fundamental mechanics of access control of operating systems. At its core , an access control model is the architectural perimeter is governs how subject (users, application or daemon processes), are permitted to interact with objects (files, memory segments, network sockets and inter-process communication channels). The core objective of these models is to enforce the principles of least privilege and maintain the integrity and confidentiality of the system environment.

The enforcement of these access control rules occurs strictly at the lowest level of the operating system hierarchy: the kernel. Modern CPU architectures are separate into privilege levels, also known as protection rings. User application operate in the restricted Ring 3, while the operating system kernel operates in Ring 0, possessing absolute, unrestricted control over the physical hardware[1]. Because user application cannot interact with hardware directly, they must request resources from the kernel via a System Call interface. The kernel acts as the ultimate, unavoidable mediator for every single access request. When a system call is invoked-such as processes attempting to open a file via the `open()` command the kernel pauses execution, evaluates the request against the active access control model, and either permits the action or terminates it with an access denial error.

Historically, the evolution of how the kernel processes these authorization decisions has been defined by two distinct and often competing paradigms : Discretionary Access Control (DAC) and Mandatory Access Control (MAC). These paradigms decide the who actually possesses the power to grant or revoke permissions. As competing environments have change from single-tenant to highly distributed, multi-tenant cloud architectures, the fundamental weaknesses of legacy DCA model have become catastrophic, forcing a system architectural shift to MAC implementations to secure the modern cyber threat landscape.

2.2 Discretionary Access Control (DAC): Mechanisms and Fatal Flaws

Discretionary Access Control, commonly referred to as DAC, is the traditional security model foundational to almost all legacy Unix and Linux operating systems. The core idea behind this model is decentralization. Instead of the operating system dictating a strict, global set of rules, the power is handed directly to the individual resource owner. If you create a file or a directory, you are granted the absolute "discretion" to decide who else is allowed to read, write, or execute it. On a technical level, this mechanism is usually implemented through those familiar rwx permission bitmasks that are tied directly to user identifiers (UIDs) and group identifiers (GIDs). It is an incredibly flexible and user-friendly system, which perfectly explains why it became the industry standard for early computing.

However, that flexibility comes with a massive security trade-off, primarily because of how DAC handles process execution. In a DAC architecture, system permissions are strictly tied to a user's identity rather than the application itself. When a user launches a program—whether that is a complex web server or just a simple PDF reader—that newly spawned process automatically inherits every single permission the executing user possesses. The system does not isolate the application authority. Therefore, if an act manage to access and exploit a vulnerability in that web server, the vulnerable process doesn't just have access to web files. It essential assumes your identity, granting the attacker freedom to control and silently modify, read, or delete any personal file you own without triggering a permission denial from the kernel.

This blind reliance on user identity also makes DAC environments incredibly susceptible to what security researchers call "confused deputy" attacks[1]. Because only checks who is requesting access (verifying user ID) rather than why they are requesting it, the system lacks true contextual awareness. In this scenario, a malicious, low privileged script essentially tricks a legitimate, highly-privileged program into misusing its authority on the attacker's behalf. The baseline OS is fundamentally incapable of recognizing that the legitimate tool is being manipulated for a malicious purpose, highlighting exactly why modern cloud and enterprise architectures had to abandon DAC for stricter security models.

2.3 The Shift to Mandatory Access Control (MAC) and Zero-Trust

Mandatory Access Control, or MAC for short, has been specifically designed to fill the immense gaps of security that the older DAC world left behind. MAC is very different—it actually removes power from individual users and file owners. Through this model, you cannot simply dictate who has access to your files; the system itself enforces a single, master security policy in-kernel[2]. You cannot bypass the system's rules, even if you created a file or have administrative rights. It is, at its core, the "adult in the room" approach to security, where the OS gets the ultimate say on all interactions.

At the kernel level, MAC is a form of zero-trust in a technical sense. The kernel, which is the core of an operating system, intervenes whenever a process (the subject) tries to access a resource such as a file or network socket (the object). It doesn't matter who you are logged in as; it only matters what the central policy says. The kernel queries a trusted database to see if that particular app has the specific clearance or "label" it needs to do that action. This provides incredibly precise control.

This degree of control is no longer merely "desirable" in modern computing; it has become a strict requirement. In a modern software-defined vehicle, you are perhaps talking about life-and-death if something goes wrong. You cannot allow a compromised IVI (the entertainment system) to communicate with the braking system or the engine controls in a car that is running Linux. Even if one part of the system is hacked, MAC serves to box an attacker in and not allow them to move across the architecture. As a result, MAC became the necessary norm of worldwide Cybersecurity standards (*like ISO/SAE 21434*). It has gone from a bespoke military tool to the bedrock of the cloud and automotive worlds.

2.4 The Linux Security Module (LSM) Architecture (Fixing TOCTOU Vulnerabilities)[3]

Although Mandatory Access Control sounds nice, it's not that simple to 'fit' into an operating system without making everything break, and it is difficult to make it work with a general-purpose toolkit. Back in the early 2000s, when the NSA originally suggested SELinux be added into the kernel, Linus Torvalds adamantly rejected locking Linux down into a single, inflexible security paradigm. What the community came up with instead was called the Linux Security Module (LSM) framework. Merged into the mainline kernel way back in version 2.6, LSM acts as a straightforward, agnostic plug-in system. Instead of forcing one model, the LSM framework provides standard plumbing that allows multiple security modules (like SELinux, AppArmor, or modern eBPF tools) to hook themselves into the kernel.

The architectural genius of LSM—if you look closely—is exactly how it solves the enormous vulnerability known as the Time-of-Check to Time-of-Use (TOCTOU) race condition. Right before LSM was standardized, early, half-baked attempts to add MAC to Linux were usually done by hijacking or wrapping the kernel's central system call table. The huge mistake with this approach was that the security validation and the actual execution were entirely separate, asynchronous activities. This left a tiny window of time between both actions. Well, that tiny delay would present a great opportunity for an advanced attacker, who could wait until the security tool authorized a benign file path, and immediately replace that benign file with a malicious symbolic link (pointing to something very sensitive, like `/etc/shadow`) right before the kernel actually opened it. Because the check had already occurred, the malicious action would succeed.

In the case of the LSM framework, inline hooking completely abolishes this race condition. Instead of verifying file paths at the top level of a user request, LSM injects generic hooks at a very deep point in the kernel execution paths. In other words, when an application asks to access a resource, the kernel will do all of the hard work first: it resolves file paths, performs normal DAC checks, and finds the actual internal memory reference. It is only at that very particular, pivotal moment—while the kernel still holds a legitimate, tamper-proof memory reference—that it invokes the LSM hook to request access permission from whatever security module happens to be active. If the security module denies the action, the kernel immediately kills off the system call. Because the check happens inline with the actual object in memory, there is absolutely zero window of time for an attacker to manipulate the target, rendering TOCTOU attacks structurally impossible.

2.5 SELinux: Complex Label-Based Type Enforcement[2]

Security-Enhanced Linux (SELinux) is typically the choice for enterprises where security must be absolute and uncompromising. SELinux (originally developed at the National Security Agency (NSA) and a part of LSM since 2001 in some form) works on what is undoubtedly one of the most strict "label-based" architectures. With this architecture, everything on the system needs to be labeled with a security context (securely tagged with a persistent cryptographic label). Every user, background process, file, directory, and even network socket gets a label.

Type Enforcement (TE) is the core engine that evaluates these labels. With this model, the centralized system policy precisely specifies which subject labels (or domains) can interact with specific object labels. It acts like a super-exclusive VIP list. As an example, if you are running an ordinary web server, that process might have a label of `httpd_t` attached to it, and the rules state explicitly that this web server is permitted to read common web files (labeled `httpd_sys_content_t`). But if there is a zero-day vulnerability that allows a hacker to take over that web server, the malicious payload will remain permanently trapped in that `httpd_t` label. If the attacker tries to read the system's sensitive password file (which is labeled `shadow_t`), the kernel steps in. Because there is no explicit rule bridging those two specific labels, the kernel silently drops the operation, completely neutralizing the attack even after the initial software was compromised.

Not just limited to file protection, SELinux has effectively become an integral part of modern cloud architecture due to its native support for Multi-Category Security (MCS). For example, if a company spins up hundreds of identical containers on one host using something like Red Hat OpenShift, SELinux automatically assigns a unique, random category string to each container's label. This guarantees flawless horizontal isolation. Two containers running with exactly the same root privileges cannot see, touch, or interact with each other's processes because they use different MCS categories.

Naturally, this architectural strictness comes with a significant price tag. Latency is introduced by checking multidimensional security contexts and resolving deep policy trees on every kernel interaction. Benchmarks typically demonstrate that using SELinux can incur a global performance penalty between 6% and 15%, with its most noticeable adverse effects on high-frequency file operations (e.g., opening or deleting directories). Moreover, since SELinux essentially needs a filesystem that supports extended attributes to make its labels persistent, it is incredibly difficult to manage over distributed storage systems, highlighting a severe trade-off between absolute security and administrative usability.

2.6 AppArmor: Usability and Path-Based Confinement[9]

Rather than tagging each and every file with complex cryptographic labels like SELinux does, AppArmor takes a completely different and vastly more pragmatic approach. Originally developed in the late 1990s and widely used as the default MAC system in Ubuntu, AppArmor is built entirely around path-based confinement. Rather than tagging the whole filesystem, it associates simple security policies (often called profiles) directly to the absolute file paths of certain executable applications. It is widely viewed as a good, easier-to-use alternative to the strictness of SELinux.

If a program that is restricted tries to do something, the kernel will intercept it at the LSM hook and check the AppArmor profile active on that particular process. These profiles are simply text files that any administrator can create, open, and edit easily. They precisely determine what an application is allowed to touch, using very fine-grained permission modes such as read, write, lock, or append. Conventionally, the most significant weakness of path-based systems was file aliasing—an attacker could create a symbolic or hard link, causing the system to grant access to a restricted file through a different "path." AppArmor's engineers solved this at the kernel level by forcing the system to resolve the true, canonical path of a file before the policy is ever evaluated, completely neutralizing link-based bypass attacks.

Instead of having to explore deep label hierarchies or look up extra information in extended filesystem attributes, AppArmor just verifies access requests against deterministic text files, which makes its performance footprint very light. It generally adds an overhead of less than 2 percent, making it an incredible fit for workstations or servers with limited computational assets. Hidden in it is a very neat "learning mode," which will sit back and watch an application run for a while, then automatically create a baseline security profile for that software, virtually eliminating the administration headache. This path-centric design, however, means it fundamentally lacks the native Multi-Category Security (MCS) capabilities that lead to SELinux's dominance in the cloud. While AppArmor easily separates containers from the host OS, keeping hundreds of individual containers strictly isolated from one another requires complex, custom configurations that are much harder to enforce out-of-the-box.

2.7 The eBPF Paradigm: Programmable, Real-Time Kernel Security[4],[10]

As strong as SELinux and AppArmor are, they share the same structural limitation: static rules. You can write a policy, load it up, and the kernel will just follow that information blindly. This legacy paradigm has been blasted to bits by the Extended Berkeley Packet Filter (eBPF). Rather than using text files or static labels that were compiled before execution, such as SELinux rulesets, eBPF embeds a very secure and sandboxed virtual machine right into the Linux kernel. This groundbreaking architecture enables developers to write custom, dynamic security scripts and load them into the OS on the fly, without the need for a system reboot or a source code patch to the core kernel.

This programmable engine was integrated into the LSM framework (which we just talked about) around Linux kernel version 5.7, creating what is known as LSM-BPF. It was a huge advancement for the security industry. Security logic could now be written in restricted C code and attached to hundreds of different system call hooks deep into the kernel through the use of a highly granular interface. However, allowing users to run arbitrary code in the high-privilege Ring 0 mode screams "recipe for total system meltdown." To stop the server from killing itself, eBPF utilizes a very strict gatekeeper called the Verifier. Before any custom bytecode is ever allowed to execute, the Verifier mathematically analyzes the program to guarantee it is absolutely safe—ensuring there are no infinite loops, no illegal memory accesses, and zero chance of triggering a kernel panic.

The real architectural value of eBPF is the ability to evaluate context dynamically in flight. Unlike a traditional MAC system that only asks one arbitrary binary question based on whether an object has a given label (yes or no), an LSM-BPF program can actually examine the entire state of the system before making a final decision. For example, if a regular user tries to take advantage of Linux namespaces to fool the system into giving them root privileges (this is usually done using the `unshare` system call), an eBPF script can instantly intercept this particular action. It is able to inspect the running task on the fly, verify what a given user is actually allowed to do, determine that this operation would constitute an illegal privilege escalation attempt, and stop the system call before the OS has even performed any resource allocations.

The programmable flexibility at this level has created an enormous new ecosystem of cloud-native security tools. There are platforms like Falco that build on eBPF mostly for deep observability—simply monitoring the system call telemetry and raising an alarm to administrators when suspicious behavior is detected. This is great, but the real frontier of this technology is where we can implement synchronous runtime enforcement, such as with tools like Cilium Tetragon. Since all the filtering logic of Tetragon is done fully in kernel space with eBPF, it does not just sit passively and watch for bad behavior. When a malicious process initiates an invalid network connection, the in-kernel eBPF logic intercepts the call

and kills the offending process within nanoseconds, completely eradicating the minuscule race conditions that plagued previous security models.

3. Future Developments in the Area

3.1 Defeating Code Reuse: Hardware-Assisted Execution Protections (PAC, BTI)[5]

Memory corruption vulnerabilities have been a constant battleground for operating systems for multiple decades. For example, when security researchers launched protections to prevent hackers from injecting their own code using mechanisms such as Data Execution Prevention (DEP), threat actors evolved. They created more sophisticated code-reuse techniques, the most notable being Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP). Rather than introducing a novel piece of malware, these attacks hijack the control flow of the application by combining relatively small and benign snippets of existing executable memory (known as "gadgets") to effect malicious behaviors. Because the underlying kernel cannot easily differentiate a true function call from a hijacked one, software-only defenses could never hold up.

Control-Flow Integrity (CFI) is currently implemented and enforced by software, but to fully prevent this class of attacks, new processor architectures place CFI directly in the hardware. One of the most notable examples is Pointer Authentication Codes (PAC), introduced by the ARMv8.3 architecture. When moving a system to 64-bit, physical addresses do not utilize all 64 available bits. PAC utilizes this vacant upper portion of the pointer itself. Prior to a function running and placing its return address on the stack, the physical CPU uses a secret hardware key to generate a cryptographic signature within those unused bits. When the function finishes and attempts to return, the CPU authenticates that signature. If an attacker has tampered with the return address in an attempt to redirect the flow to a malicious ROP gadget, the signature verification fails, and the processor instantly triggers a fault, killing the compromised process before the jump can occur.

Though PAC offers a huge defense layer for return addresses, it cannot be the entire solution to indirect branching. To close the remaining gaps, silicon vendors introduced Branch Target Identification (BTI) in the ARMv8.5 instruction set. BTI enforces a non-negotiable "landing pad" at the hardware level. In a typical JOP (Jump-Oriented Programming) attack, an attacker tries to compel the CPU to jump into the center of a random function in order to execute a particular instruction. With BTI enabled, the processor hardware categorically refuses to execute an indirect jump unless the target address contains a particular "landing pad" instruction generated by the compiler. If the attacker attempts to redirect the flow to a random location in memory, the CPU simply rejects the branch.

When PAC and BTI are combined, they represent a monumental leap in foundational cybersecurity. They effectively strip attackers of the ability to weaponize existing memory, shifting the burden of memory protection from the overworked operating system kernel directly down to the unyielding logic of the physical processor.

3.2 Hardware-Assisted eBPF Isolation (The HIVE Framework)[8]

Though we sang the praises of eBPF's revolutionary capabilities in Chapter 2, it still has a single critical Achilles' heel. As eBPF programs run in-kernel, they exist with the same threat model as the OS. Once an APT gets full control in Ring 0 with root access, they could just reach into the kernel's memory and tear out the eBPF infrastructure. A seasoned attacker could simply export the security filters and disable them as well, or edit the eBPF data maps to hide evidence of their activity, or render the telemetry tools completely blind. In short, if the host kernel goes down, so does the security monitor.

This basic weakness was later tackled by the most advanced security research with a concept of hardware-assisted eBPF isolation, such as in architectures like HIVE. HIVE aims to maintain the eBPF execution engine but decouples it from the vulnerable kernel space physically. HIVE, instead of trusting the host OS to do right by its own security tools, uses hardware-backed Trusted Execution Environments (TEEs) or specialized virtualization extensions to lock down the eBPF runtime.

In this sandboxed model, the host kernel still invokes the eBPF hooks when their corresponding system calls are made, but the actual security logic, including data maps and even the Verifier, runs in a completely separate hardware-encrypted domain. That makes for an intriguing twist in the power dynamics of these systems. Even if a threat actor successfully installs a kernel-level rootkit, they are locally unable to change the eBPF bytecode or modify security policies. Even if the malicious OS tries to turn off the eBPF monitor, the actual hardware refuses that instruction. With HIVE and others converting eBPF from a kernel-level software hole to an immune, hardware-backed guardian, modern cloud-native security architectures are closing their last significant weak link.

3.3 Confidential Computing in Cloud Architectures (Intel TDX / AMD SEV-SNP)[6]

So far, all the security models we have examined—even highly sophisticated frameworks like eBPF—have exhibited one fatal flaw: they assume that the operating system kernel is, indeed, trustworthy. If a sophisticated attacker can break through a hardware weakness or a zero-day vulnerability to gain unbridled Ring 0 access, software-based mechanisms (like SELinux or AppArmor) essentially lose their force. This ultimate threat can be countered by moving the battleground away from the operating system and embedding the security directly into physical silicon in this next decade of cybersecurity. This incipient, hardware-centric paradigm is popularly referred to as Confidential Computing.

At the heart of this technological transformation is the Trusted Execution Environment (TEE). Major processor manufacturers have introduced hardware-level security extensions, like Intel's Software Guard Extensions (SGX) and AMD's Secure Encrypted Virtualization (SEV). These empower developers to create highly isolated, hardware-encrypted regions of physical memory that we call enclaves. On-the-fly encryption and decryption are performed by the physical CPU when a sensitive application running inside one of these enclaves accesses data, protecting that data while it is being actively processed in memory.

This forms a practically unbreakable zero-trust architecture at the hardware layer. Not only is it resistant to malware, but even if a threat actor managed to completely compromise the underlying host OS and the Linux kernel, successfully hack the hypervisor, or gain physical access to the server rack in the data center itself, they still cannot read or modify any data running within an enclave. When a nefarious root process attempts to peek into the enclave's memory space, it won't matter: all they will see is random, encrypted garbage. Relying on just the OS to keep data safe can no longer be enough as cloud computing environments grow and command ever greater degrees of sensitive information. The ultimate solution will be the hardware-enforced isolation of enterprise workloads.

3.4 In-Kernel Machine Learning and Automated Anomaly Detection

On the other hand, hardware enclaves such as Intel SGX solve the physical security problem, but when you scale them to modern cloud-native environments, the thousands of nodes create a monstrous administrative bottleneck. Typically, in a contemporary Kubernetes cluster, thousands of microservices could be spun up and talk to one another before terminating in seconds. At the speed that level of infrastructure is changing, it is physically impossible for a human cybersecurity team to write SELinux labels or eBPF scripts manually fast enough, test them, and then deploy them. The next evolution of operating system security, and in fact one of the last steps needed to close this gap for good, is moving Machine Learning (ML) out of its bubble and right into the access control pipeline on an OS-level—enabling fully autonomous zero-trust architectures.

So far, the application of AI in cybersecurity was limited to user-space analytics. Systems would kick out thousands of kernel logs, ship them all to an outside Security Information and Event Management (SIEM) dashboard, so that an AI could flag suspicious behavior for a human to review the next morning. But with the future come lightweight ML models that operate much closer to the kernel. With the assistance of deep observability in the LSM-BPF framework we talked about earlier, machine learning algorithms can now directly run on top of the system call interface, where they passively receive execution telemetry as it occurs.[7],[11]

Where human-written static policies fall short, those ML models use their time to learn the precise behavioral baseline of each application—what files a web server normally touches, and what network sockets it typically opens. In the case of a zero-day exploit and an attendant anomalous web server behavior (e.g., making attempts to spawn a hidden bash shell), the AI does not merely send out an alert. Instead, it instantaneously composes a bespoke eBPF filter, runs it through the Verifier in the kernel, and autonomously denies system calls within nanoseconds. This is the epitome of access control evolution. We have moved from the legacy DAC era where users guessed at file permissions, through the MAC era of rigid administrator rules, and are now entering a future where the operating system intelligently and dynamically defends itself.

4. Conclusion

4.1 Summary of the Evolution

The journey of operating system access controls has been a continuous fight between new ideas and an ever-changing threat landscape. For a time, the industry used Discretionary Access Control, a model that was made for a simpler time when computers were not as complicated and it was okay to trust a user's identity. However, as companies' computer systems became more complex and attackers found ways to take control of legitimate processes, the problems with user-centric permissions became clear and a big change was needed.

This important realization led to the use of Mandatory Access Control. Frameworks like SELinux and AppArmor, which work well with the Linux Security Module architecture, changed the way things were done by enforcing policies that no user or compromised application could get around. While these models provided isolation and stopped problems like TOCTOU, their strict and unchanging nature had a hard time keeping up with the fast-paced and always-changing world of modern cloud computing.

The introduction of the Extended Berkeley Packet Filter was a step forward from static rules to rules that can be changed in real time. By letting engineers add secure code directly into the kernel, the cybersecurity industry was finally able to keep up with the speed and detail needed to secure large Kubernetes clusters. Now, as we look to the future, it is clear that even the best software needs to be backed up by hardware. The ongoing use of hardware to protect execution, Confidential Computing enclaves, and autonomous Machine Learning models is the next step in this journey. We are moving away from operating systems that just follow rules made by people and towards environments that can defend themselves with the help of hardware. Operating system access controls are getting better and better. Operating system access controls are becoming more intelligent. Operating system access controls are the key to a secure computer system.

4.2 Recommendations for Future Research

The move to security systems that use hardware and work on their own is a promising direction. However, there are some important areas that need to be looked at more closely by universities and companies.

First, we need to ensure that hardware-assisted eBPF isolation frameworks are standardized. Currently, research like the HIVE framework is still in the early stages. We need to create a standard that works for all companies and does not favor one vendor over others. This standard should make it possible to offload security work from the kernel to Trusted Execution Environments (TEEs) so that large enterprises can adopt these systems. We should also focus on reducing the latency it takes for the kernel and these secure areas to communicate. This is vital so that the system remains safe without sacrificing overall performance.

Second, we must be very careful when adding Machine Learning to the kernel. We have to consider how malicious actors might try to trick the system through adversarial attacks. They might try to "poison" the behavioral baseline so they can perform unauthorized actions without being detected. We need to ensure that the pipelines used to train these in-kernel ML models are tamper-proof and secure.

Lastly, as we move toward a "Zero-Trust-on-Silicon" model, we need to figure out how to manage and patch security keys that are built into the hardware. Solving these problems is the final step in creating an operating system that is truly resilient and capable of defending itself in the cloud-native era. By focusing on the standardization of eBPF isolation, the security of in-kernel ML, and hardware key management, we can build a foundational defense system that is both safe and efficient.

5. References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
- [2] P. Loscocco and S. Smalley, "Integrating Flexible Support for Security Policies into the Linux Operating System," *Proceedings of the FREENIX Track: 2001 USENIX Annual Technical Conference*, Boston, MA, 2001, pp. 29-42.
- [3] C. Wright, C. Cowan, S. Smalley, J. Morris, and G. Kroah-Hartman, "Linux Security Modules: General Security Support for the Linux Kernel," *Proceedings of the 11th USENIX Security Symposium*, San Francisco, CA, 2002.
- [4] B. Gregg, *BPF Performance Tools*. Boston, MA: Addison-Wesley Professional, 2019.
- [5] K. Kurokawa et al., "Hardware-Assisted Control-Flow Integrity for ARM Core," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2102-2115, 2021.
- [6] V. Costan and S. Devadas, "Intel SGX Explained," *Cryptology ePrint Archive*, Report 2016/086, 2016. [Online]. Available: <https://eprint.iacr.org/2016/086>
- [7] M. Peuster and S. Karl, "Understand your containers: eBPF-based deep observability for cloud-native applications," *IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, Leganes, Spain, 2020, pp. 1-6.
- [8] D. J. Tian et al., "HIVE: Hardware-Assisted Isolation of eBPF," *Proceedings of the 28th ACM Conference on Computer and Communications Security (CCS)*, Virtual Event, 2021, pp. 123-137.
- [9] "AppArmor Core Documentation," *Ubuntu Community Help Wiki*. [Online]. Available: <https://help.ubuntu.com/community/AppArmor>. [Accessed: Apr. 27, 2026].
- [10] Cilium Authors, "Tetragon: eBPF-based Security Observability and Runtime Enforcement," *Cilium Documentation*. [Online]. Available: <https://cilium.io/projects/tetragon>. [Accessed: Apr. 27, 2026].
- [11] A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153-1176, 2016.